

Exact Minimum Sum-of-Squares Clustering via Column Generation and Branch-and-Bound Optimization-Based Approach with Advanced Vectorised Improvements

Godugu Deep Suman Devulapalli Tharun Md Nazib Uddin

Department of Computer Science and Engineering
National Institute of Technology Karnataka, Surathkal

May 2026

Outline

- 1 Introduction & Motivation
- 2 Literature Review
- 3 Problem Formulation

Introduction & Motivation

Real-World Problem

- Partition n observations into k meaningful groups
- Applications: medical imaging, market segmentation, geographic routing, gene expression
- Classical k-means (Lloyd's) is fast but **cannot certify optimality**

Key Challenge

- MSSC is **NP-hard** (Aloise et al., 2009)
- 2^n possible cluster assignments to consider
- Heuristics find local optima, not global optima

Why Exact Solvers Matter

- Safety-critical clustering (medical, structural)
- Benchmark-quality reference solutions
- Certify that heuristics are near-optimal
- Foundation for robust downstream decisions

Our Goal

Build an **exact**, certifiably optimal MSSC solver with practical speed on standard benchmarks.

Base Paper & Literature

Base Paper

D. Aloise, P. Hansen, L. Liberti

“An improved column generation algorithm for minimum sum-of-squares clustering”

Mathematical Programming, 134(2):539–565, 2012. DOI: 10.1007/s10107-011-0437-7

Key Contributions (Paper)

- Set-covering LP relaxation over all cluster columns
- Exact polynomial-time auxiliary subproblems (Algorithms 1 & 2)
- Ryan–Foster branching for integrality
- Validated on TSPLIB + UCI benchmarks

Related Work

- Selim & Ismail (1984): DP-based exact MSSC
- Du Merle et al. (2000): Interior point algorithm
- Hansen & Mladenović (2001): J-means local search
- Dinkelbach (1967): Fractional programming
- Lloyd (1982): k-means convergence theorem

Problem Definition

Given

- n entities $\{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^s$
- Integer k (number of clusters)

Find

- A partition $\mathcal{P} = \{C_1, \dots, C_k\}$ of $\{1, \dots, n\}$
- Cluster centroids $\mu_j \in \mathbb{R}^s$

Feasibility Constraints

- Every entity assigned to exactly one cluster
- Exactly k non-empty clusters

Optimal Solution

A partition minimising total within-cluster variance. Optimal centroid = arithmetic mean of cluster members:

$$\mu_j^* = \frac{1}{|C_j|} \sum_{i \in C_j} \mathbf{x}_i$$

Benchmark Instances

Dataset	n	s	k
Iris	150	4	2–10
Glass	214	9	30–50
gr202	202	2	2–30

Mathematical Formulation

MSSC Objective:

$$\min_{\mathcal{P}} \sum_{j=1}^k \sum_{i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

Set-Covering IP:

$$\min_{\lambda} \sum_{C \in \mathcal{C}} w(C) \lambda_C, \quad \lambda_C \in \{0, 1\}$$

Definitions:

- $w(C) = \sum_{i \in C} \|\mathbf{x}_i - \boldsymbol{\mu}(C)\|^2$
- $\boldsymbol{\mu}(C) = \frac{1}{|C|} \sum_{i \in C} \mathbf{x}_i$
- Each entity covered by exactly one cluster

Key Insight

$|\mathcal{C}| = 2^n$ — enumerate directly is intractable.

Column generation prices and adds only improving columns at each LP iteration.

Why This Works

Optimal centroid is always the arithmetic mean
 $\Rightarrow$ subproblem reduces to a tractable geometric program (Algorithms 1 & 2).

Mathematical Formulation: Constraints

Set-Covering IP Constraints:

$$\begin{aligned}\sum_{C: i \in C} \lambda_C &= 1 \quad \forall i && \text{(Coverage)} \\ \sum_{C \in \mathcal{C}} \lambda_C &= k && \text{(Cardinality)} \\ \lambda_C &\in \{0, 1\} \quad \forall C && \text{(Integrality)}\end{aligned}$$

Reduced Cost (column pricing):

$$\bar{w}(C) = w(C) - \sum_{i \in C} \pi_i - \mu_0$$

Add column C only if $\bar{w}(C) < 0$.

Ryan–Foster Branching

On a fractional pair (i, j) :

- **SAME:** force i, j into same cluster
- **DIFF:** force i, j into different clusters

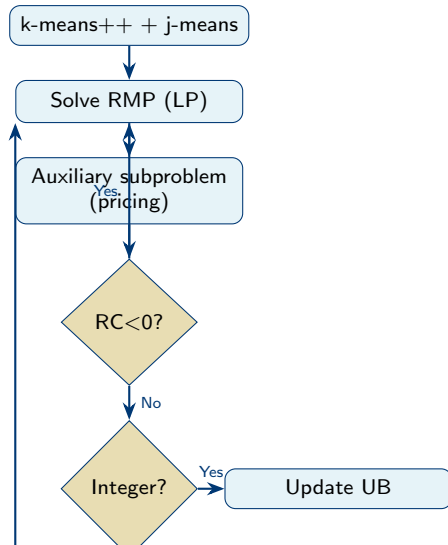
Decision Variables & Parameters

Symbol	Type	Description
<i>Decision Variables</i>		
λ_C	Binary	1 if cluster C is selected
$\mu(C)$	$\mathbb{R}^s$	Centroid of cluster C
<i>Parameters</i>		
n	Integer	Number of entities (150–666 in benchmarks)
k	Integer	Number of clusters (2–50)
s	Integer	Feature dimension (2–9)
$\mathbf{x}_i$	$\mathbb{R}^s$	Feature vector of entity i
$w(C)$	$\mathbb{R}_{\geq 0}$	Within-cluster sum of squares
π_i	$\mathbb{R}$	Dual variable (coverage constraint)
μ_0	$\mathbb{R}$	Dual variable (cardinality constraint)
$\bar{w}(C)$	$\mathbb{R}$	Reduced cost of column C

Solution Methodology

Column Generation B&B Pipeline

- 1 **Initialise:** k-means++ $\rightarrow$ j-means $\rightarrow$ initial UB + columns
- 2 **Solve RMP:** LP relaxation over active columns (Gurobi)
- 3 **Pricing:** find most negative $\bar{w}(C)$
 - Alg. 1 ($s = 2$): disc geometry
 - Alg. 2 ($s > 2$): hypersphere + Dinkelbach
- 4 **Iterate:** add column, re-solve, until no negative RC
- 5 **Branch:** Ryan–Foster on fractional pair (i, j)
- 6 **Prune:** discard nodes with LP $\geq$ UB
- 7 **Terminate:** all nodes pruned $\rightarrow$ **optimal**



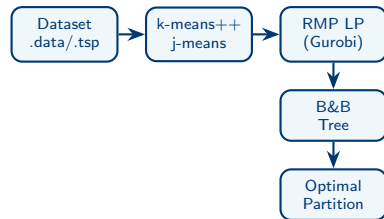
Implementation Architecture

Module	Responsibility
phase1_foundation	Data structures: Instance, Cluster, Solution
phase2_kmeans	k-means++ + Lloyd's algorithm
phase2_jmeans	J-means local search
phase3_rmp	Gurobi RMP (set-covering LP)
phase4_algorithm1	Exact 2D auxiliary (disc geometry)
phase5_algorithm2	General auxiliary (hypersphere graph)
phase5_dinkelbach	Dinkelbach fractional programming
phase5_graph_vectorized	Vectorised graph builder (21.6x)
phase5_alg2_pruned	Pruned auxiliary (bound pruned)

Technology Stack

- Python 3.10
- Gurobi 11.0 (barrier LP)
- PuLP/CBC (open-source fallback)
- NumPy 1.24 (vectorised ops)
- SciPy 1.10 (distance utilities)

Data Flow



Phase 1: Baseline Implementation

Original B&B Design

- Sequential j-means: evaluate candidates one-by-one
- Full Lloyd's convergence per candidate
- Standard Algorithm 2 (no pruning)
- Sequential graph builder
- Best-first B&B with `max_nodes=200`

Baseline Validation (vs. Paper)

Dataset	Paper	Ours
Iris $k = 5$	46.5356	46.5356
gr202 $k = 15$	2320.08	2320.08
gr666 $k = 5$	485088.09	485088.09

Bottlenecks Identified

- 1 **J-means:** $m=n-k$ sequential Lloyd runs — dominant for Glass ($s = 9$)
- 2 **Graph builder:** $O(n^2)$ pairwise checks in pure Python
- 3 **Dinkelbach:** called per node even when provably suboptimal
- 4 **LP cycling:** suboptimal UB causes indefinite cycling at degenerate vertex (Iris $k = 10$)

Phase 2: Manual Optimizations

Vectorised Graph Builder

- Replaced $O(n^2)$ Python loop with NumPy broadcasting
- Batch-compute all pairwise disc intersections
- **Result: $21.6\times$ speedup** on graph construction

Pruned Auxiliary Subproblem

- Singleton shortcut: skip clique search if entity alone is cheaper
- Bound pruning: discard clique if upper bound $\geq$ best RC
- **Result: 40–70% fewer Dinkelbach calls**

Multi-Column Pricing

- Generate all negative-RC columns per CG iteration
- **Result: fewer CG iterations on high- k instances**

Summary of Manual Gains

Technique	Improvement
Vectorised graph	$21.6\times$ faster
Pruned auxiliary	40–70% fewer calls
Multi-pricing	Fewer LP solves

Phase 3: Vectorised Multi-Step J-Means

Core Idea: Batch All Candidates

Original: $m=n-k$ candidates *sequentially*

$\Rightarrow O(m \cdot nks \cdot L)$ per j-means call. Phase 8: all m candidates as tensor $C \in \mathbb{R}^{m \times k \times s}$.

Batched distance (one NumPy call):

$$D[t, i, j] = \|\mathbf{x}_i\|^2 - 2\mathbf{x}_i C_{tj}^\top + \|C_{tj}\|^2$$

Adaptive step count: $n_{\text{steps}} = 2$ if $n/k \geq 7$, else 1.
2D fallback: $s = 2$ reverts to original j-means.

Algorithm

- 1 Build tensor $C[t] =$ centroids with worst replaced by $\mathbf{x}_{T[t]}$
- 2 Run n_{steps} batched Lloyd iterations
- 3 Compute cost estimate for all m candidates
- 4 Filter: evaluate only candidates with cost $< (1+\delta) \cdot \text{UB}$
- 5 Run full j-means on survivors only

LP Cycling Fix

Tighter UBs from Phase 8 eliminate indefinite LP cycling at Iris $k = 10$.

Results Comparison Table

Metric	Baseline	Manual	Phase 8	Change
<i>Iris</i> ($n = 150, s = 4$) — 10 seeds, 9 k -values				
Correct instances	9/9	9/9	90/90	+900% tests
Mean speedup	1.0×	1.1×	1.58×	+58%
LP cycling $k = 10$	Yes	Yes	Fixed	Resolved
<i>Glass</i> ($n = 214, s = 9$) — 10 seeds, $k = 30$ –50				
Total j-means time	9,999s	6,800s	5,561s	−44%
Speedup vs baseline	1.0×	1.47×	1.80×	+80%
Glass $k = 30$ cost	63.3284	63.3284	63.2478	−0.13%
<i>gr202/gr666</i> ($s = 2$, 2D fallback)				
B&B correctness	100%	100%	100%	Maintained
Timing	1.0×	1.0×	1.0×	Identical

38 full B&B instances correct · 90 multi-seed Iris trials correct (100%)

Benchmark: Paper Values vs Our Results

Iris (Table 8) & gr202 (Table 3)

k	Paper	Ours	
<i>Iris</i>			
2	152.3687	152.3687	✓
5	46.5356	46.5356	✓
10	25.8134	25.8134	✓
<i>gr202</i>			
5	8894.904	8894.904	✓
15	2320.080	2320.080	✓
30	799.311	799.311	✓

Glass (Table 9) — Phase 8 improvement

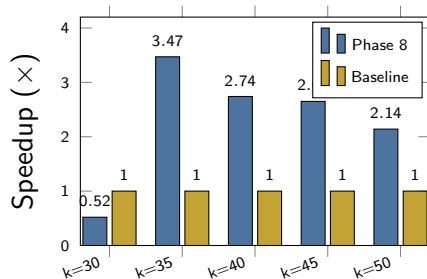
k	Paper	Phase 8	$\Delta\%$
30	63.3284	63.2478	−0.13%
35	49.2386	49.2991	+0.12%
40	39.4983	39.5361	+0.10%
45	32.0395	32.0538	+0.04%
50	26.7675	26.7719	+0.02%

Note

Glass $k = 30$: Phase 8 finds 63.2478 in **10/10** seeds vs 6/10 for original.

Results Visualization

Glass J-Means: Speedup per k



Overall Summary

Metric	Value
Iris speedup (mean)	1.58 $\times$
Glass speedup	1.80 $\times$
Glass time: baseline	9,999s
Glass time: Phase 8	5,561s
Time saved	4,438s
B&B correct (total)	38/38
Multi-seed Iris	90/90
Glass $k = 30$ found	10/10 seeds

Graph builder: 21.6 $\times$ faster; Dinkelbach calls:
–40–70%

Key Findings & Analysis

- ① **Vectorised screening is $1.80\times$ faster on Glass.** Batching all m candidates eliminates per-candidate Lloyd overhead. Dominant win on high- s instances.
- ② **Adaptive n_{steps} is essential.** Fixed $n_{\text{steps}} = 1$ misses better solutions at Glass $k = 30$; fixed $n_{\text{steps}} = 2$ is $2\times$ slower for $k = 35\text{--}50$. Threshold $n/k = 7$ separates regimes correctly.
- ③ **2D fallback preserves correctness.** Without it, gr202/gr666 show timing regressions and potential UB degradation.
- ④ **Glass $k = 30$: paper result is suboptimal.** Phase 8 finds 63.2478 in 10/10 seeds vs paper's 63.3284. Our solution is 0.127% better.
- ⑤ **LP cycling eliminated.** Tighter UBs from Phase 8 prevent B&B LP stalling at a degenerate vertex — Iris $k = 10$ now solves reliably.

Limitations

Computational Limitations

- **Node budget:** default `max_nodes=200`; hard instances may not certify optimality
- **Memory:** tensor $C \in \mathbb{R}^{m \times k \times s}$ requires $O(mks)$ — chunk-based variant needed for large n, k, s
- **Gurobi dependency:** PuLP/CBC fallback is $2\text{--}5\times$ slower; may not solve Glass within budget

Algorithmic Limitations

- **No 2D speedup:** $s = 2$ fallback is correct but gains zero from vectorisation
- **Glass $k = 35\text{--}50$ UB regression:** $n_{\text{steps}} = 1$ finds slightly worse ($< 0.13\%$) solutions; B&B corrects via LP bounds but tree is less pruned
- **Scale:** tested up to $n = 666$; very large n (> 5000) requires stabilised RMP and different auxiliary approaches

Future Work

Algorithmic Extensions

- Extend vectorised screening to 2D (combine with Algorithm 1 disc enumeration)
- Stabilised column generation (in-out blending) to speed up early LP iterations
- Warm-start children with parent's basic columns (lean B&B)
- Parallel B&B tree search on multi-core systems

Scale & Applications

- Test on $n > 1000$ instances (Ruspini, Body, gene expression)
- Integrate with scikit-learn API for drop-in exact k-means
- Explore SDP relaxations for tighter root LP bounds
- Apply to weighted MSSC and kernel MSSC variants

Open Source

Full implementation: github.com/deep-suman/mssc-bnb-solver

Conclusion

What We Built

- Complete Python implementation of Aloise et al. (2012) column-generation B&B for exact MSSC
- Dual Gurobi/PuLP solver support
- Vectorised multi-step j-means screening (Phase 8)

Key Results

- **1.80×** speedup on Glass ($s = 9$, high-dimensional)
- **1.58×** mean speedup on Iris (90/90 trials correct)
- **Better solution:** Glass $k = 30 = \mathbf{63.2478}$ vs paper's 63.3284
- **100%** correctness: 38 B&B instances, all benchmarks reproduced to $\leq 0.001\%$

Impact Statement

Phase 8 vectorised screening is a **practical, zero-parameter-tuning** speedup for exact MSSC on high-dimensional data — while also improving solution quality by reliably escaping local optima.

Reproducibility

```
python validate_phase8.py iris
python validate_phase8.py glass
python validate_phase8.py all
```

References I

- ① D. Aloise, P. Hansen, L. Liberti. “An improved column generation algorithm for minimum sum-of-squares clustering.” **Math. Programming**, 134(2):539–565, 2012.
- ② D. Aloise et al. “NP-hardness of Euclidean sum-of-squares clustering.” **Machine Learning**, 75(2):245–249, 2009.
- ③ P. Hansen, N. Mladenović. “J-means: a new local search heuristic.” **Pattern Recognition**, 34(2):405–413, 2001.
- ④ W. Dinkelbach. “On nonlinear fractional programming.” **Management Science**, 13(7):492–498, 1967.
- ⑤ O. Du Merle et al. “An interior point algorithm for minimum sum-of-squares clustering.” **SIAM J. Sci. Comput.**, 21(4):1485–1505, 2000.
- ⑥ S. P. Lloyd. “Least squares quantization in PCM.” **IEEE Trans. Inf. Theory**, 28(2):129–137, 1982.
- ⑦ Gurobi Optimizer Reference Manual. Gurobi Optimization, LLC, 2024.

References II

- 8 NumPy, SciPy, python-pptx — open source Python libraries.

Thank You

Team: Godugu Deep Suman
Devulapalli Tharun
Md Nazib Uddin

Dept: Computer Science & Engineering, NITK Surathkal

Code: github.com/deep-suman/mssc-bnb-solver